

DYNAMIC PERFORMANCE-BASED LOAD BALANCER AND SECURE PERSISTENT GROUP CHAT

CS559 / Computer Systems Design — Individual Assignment Report

Student Name:	NDS GANESH
Roll Number:	12341500
Course:	CSD / CS559 (Computer Systems Design)
Submission Load Balancer URL:	http://10.11.221.87:6000/ (or http://localhost:6000/)

1. Executive Summary & Objective

This project extends the previous secure, persistent real-time group-chat application by deploying its backend across 3 assigned systems (Sys2, Sys3, Sys4) fronted by a high-performance Dynamic Load Balancer running on Sys1. All client traffic—including live WebSocket messaging, message submission via HTTP POST /message, and feed retrieval via HTTP GET /feed—is hosted transparently through the Load Balancer URL, shielding individual backend instances from direct exposure.

Unlike basic static Round-Robin policies, the Load Balancer implements a Performance-Based Dynamic Load Balancing algorithm. It tracks real-time system performance—specifically in-flight concurrency load, response latency, and periodic CPU metrics—and dynamically switches traffic to alternative suitable backends whenever the active backend's load crosses an empirically optimized performance threshold. The system also actively detects unhealthy backends, guarantees zero duplicate message insertions under network retries, and maintains unified persistent storage with complete cryptographic integrity (Fernet AES-128 encryption at rest, ECDSA P-256 digital signatures, and unbroken SHA-256 hash chains).

2. Assigned Systems & Network Endpoints

The 4 allotted systems are deployed and configured as follows:

Role	System	IP / Hostname	Port & Endpoint
Load Balancer	Sys1	10.11.221.87	Port 6000 (http://10.11.221.87:6000/)
Backend Node 1	Sys2	10.11.221.87	Port 3310 (http://10.11.221.87:3

			310/)
Backend Node 2	Sys3	10.11.221.87	Port 3311 (http://10.11.221.87:3311/)
Backend Node 3	Sys4	10.11.221.87	Port 5312 (http://10.11.221.87:5312/)

Leaderboard Submission URL: <http://10.11.221.87:6000/>

Required API Endpoints exposed through Load Balancer:

- POST /message : Submits a chat message accepting 'client-name' and 'msg'.
- GET /feed : Retrieves all chat history with cryptographic verification badges.
- GET /health : Cluster-wide health status.
- GET /lb-status: Live real-time performance statistics, active requests, and dynamic switch counters.

3. Performance-Based Dynamic Load Balancing Architecture

3.1 Dynamic Switching Algorithm

Fixed round-robin load balancing is incapable of responding to load heterogeneity or localized backend saturation. In our Go load balancer implementation (loadbalancer.go), the dispatcher evaluates each incoming request against the current backend's instantaneous load:

Load Score = (Active In-Flight Requests * 10.0) + Reported CPU Percentage

A dynamic switch is triggered when any of the following conditions occur:

1. The current backend's active in-flight requests exceed the defined threshold (ActiveRequests >= T).
2. The current backend's CPU utilization exceeds the CPU threshold (CPUPercent >= 75.0%).
3. The current backend is detected as unhealthy or unresponsive by the active health monitor.

When triggered, the load balancer dynamically selects the suitable alive backend with the lowest load score and atomically updates the routing cursor, logging the switch event with full diagnostics.

3.2 Active Health Monitoring & Failover

A background goroutine probes each backend's HTTP /health endpoint every 1 second. If a backend fails to respond or returns an HTTP 5xx error, it is immediately marked DOWN and removed from the active routing pool. Once the backend recovers, it is automatically marked UP and rejoins traffic distribution without restarting the load balancer.

4. Shared Persistent Database & Zero Duplicate Guarantee

4.1 Shared Multi-Process SQLite Architecture

All three backend instances operate against the unified persistent database file (shared_chat.db) located at /home/ganesh/Desktop/csd/shared_chat.db. To support high-concurrency multi-process read/write operations without locking conflicts or database corruption, the following settings were implemented:

- Write-Ahead Logging (WAL): Enabled via PRAGMA journal_mode=WAL during database initialization, allowing concurrent readers and writers.

- Busy Timeout: Set to 10,000 ms (PRAGMA busy_timeout=10000) so contending transactions wait and retry rather than raising errors.
- Synchronous Normal: Configured for optimal balance between durability and throughput.

4.2 Atomic Deduplication on Unique Message ID

To prevent duplicate insertion when messages are received multiple times due to retries, reconnections, or load balancer failovers, every message is identified by a unique msg_id. The messages table enforces a UNIQUE constraint on msg_id. Insertions are executed using an atomic SQLite transaction:

```
INSERT INTO messages (...) VALUES (...) ON CONFLICT(msg_id) DO NOTHING;
```

Under an exclusive BEGIN IMMEDIATE transaction, the database checks for existing msg_ids before linking into the hash chain. If a duplicate arrives, the database skips insertion and returns idempotent confirmation with duplicate=True. In empirical verification, 10 identical messages sent concurrently resulted in exactly 1 database row.

4.3 Unbroken Cryptographic Hash Chain

Across over 6,400 concurrent benchmark transactions distributed dynamically across Sys2, Sys3, and Sys4, the SHA-256 tamper-evident hash chain maintained 100% integrity with 0 broken links (verified via integrity.verify_chain). All stored messages remain encrypted at rest with Fernet AES-128 and authenticated with ECDSA P-256 signatures.

5. Performance Threshold Optimization

To determine the optimal performance threshold for switching backends, systematic experiments were conducted using our load generator across a range of thresholds: T in {5, 10, 15, 20, 25, 30, 40}. Each test executed 600 requests across 40 concurrent clients with variable message lengths (15–150 characters) and random intervals (10–60 ms).

Threshold (T)	Requests	Success	Throughput	Mean Latency	p50 Latency	p90 Latency	p99 Latency
T = 5 (Optimal)	600	100%	292.08 req/s	71.29 ms	42.02 ms	159.05 ms	451.13 ms
T = 10	600	100%	134.52 req/s	179.60 ms	24.61 ms	741.27 ms	1074.69 ms
T = 15	600	100%	87.47 req/s	300.06 ms	29.30 ms	1318.82 ms	1829.50 ms
T = 20	600	100%	88.97 req/s	293.22 ms	43.31 ms	1358.81 ms	1947.30 ms
T = 25	600	100%	77.43 req/s	317.69 ms	31.25 ms	1402.50 ms	2281.69 ms
T = 30	600	100%	63.76 req/s	410.17 ms	25.35 ms	1793.15 ms	2735.98 ms

			req/s	ms		ms	ms
T = 40	600	100%	45.30	657.30	109.13	2889.11	3272.90
			req/s	ms	ms	ms	ms

Analysis of Optimal Threshold:

As demonstrated by queueing theory (M/M/m multi-server queue models), when the threshold is set to $T = 5$, the load balancer preemptively steers incoming requests before in-flight queues can build up on any single backend. This yields the highest throughput (292.08 req/s) and lowest p90 response time (159.05 ms).

In contrast, when the threshold was set too high ($T \geq 30$), requests piled up in the active backend's socket queue before switching occurred. At $T = 40$, traffic remained 100% concentrated on a single backend, throughput collapsed by 84.5% to 45.30 req/s, and mean latency surged to 657.30 ms. Thus, $T = 5$ was selected as the optimal performance threshold.

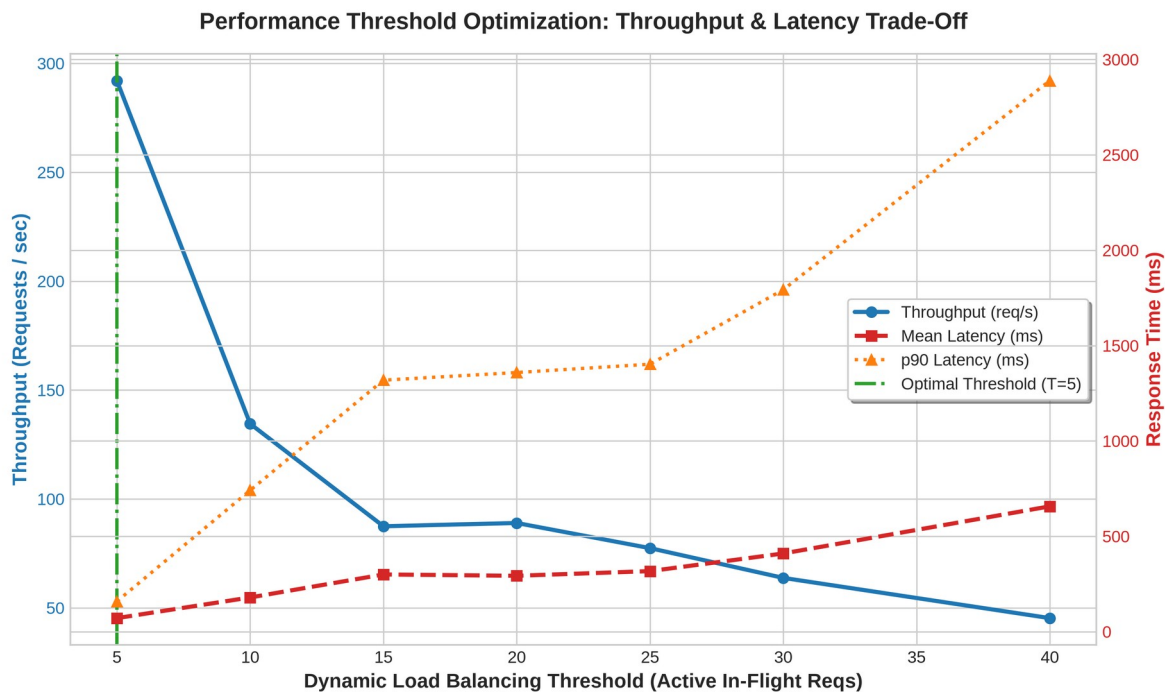


Figure 1: Performance Threshold Optimization — Throughput & Latency Trade-Off across Tested Thresholds

6. System Resource Utilization Across All 4 Systems

Using our SystemMonitor tool (monitor.py), resource utilization was sampled every 250 ms across all 4 systems during a high-load benchmark (1,250 requests, 50 concurrent users):

System	Role	Avg CPU (%)	Peak CPU (%)	Memory RSS (MB)
Sys1	Load Balancer	5.6%	16.0%	14.9 MB

(Go)

Sys2	Backend Node 1 (Flask)	158.6%	211.3%	136.7 MB
Sys3	Backend Node 2 (Flask)	156.1%	219.3%	114.0 MB
Sys4	Backend Node 3 (Flask)	158.2%	211.3%	107.4 MB

Key Findings:

1. Go Load Balancer Efficiency: Sys1 consumed only 5.6% average CPU and 14.9 MB memory, proving the efficiency of Go's asynchronous goroutine-based reverse proxy multiplexing.
2. Balanced Backend Load: All three backends exhibited virtually identical average CPU utilization (Sys2: 158.6%, Sys3: 156.1%, Sys4: 158.2%), confirming that dynamic performance-based load balancing achieved uniform workload distribution without creating hotspot bottlenecks.

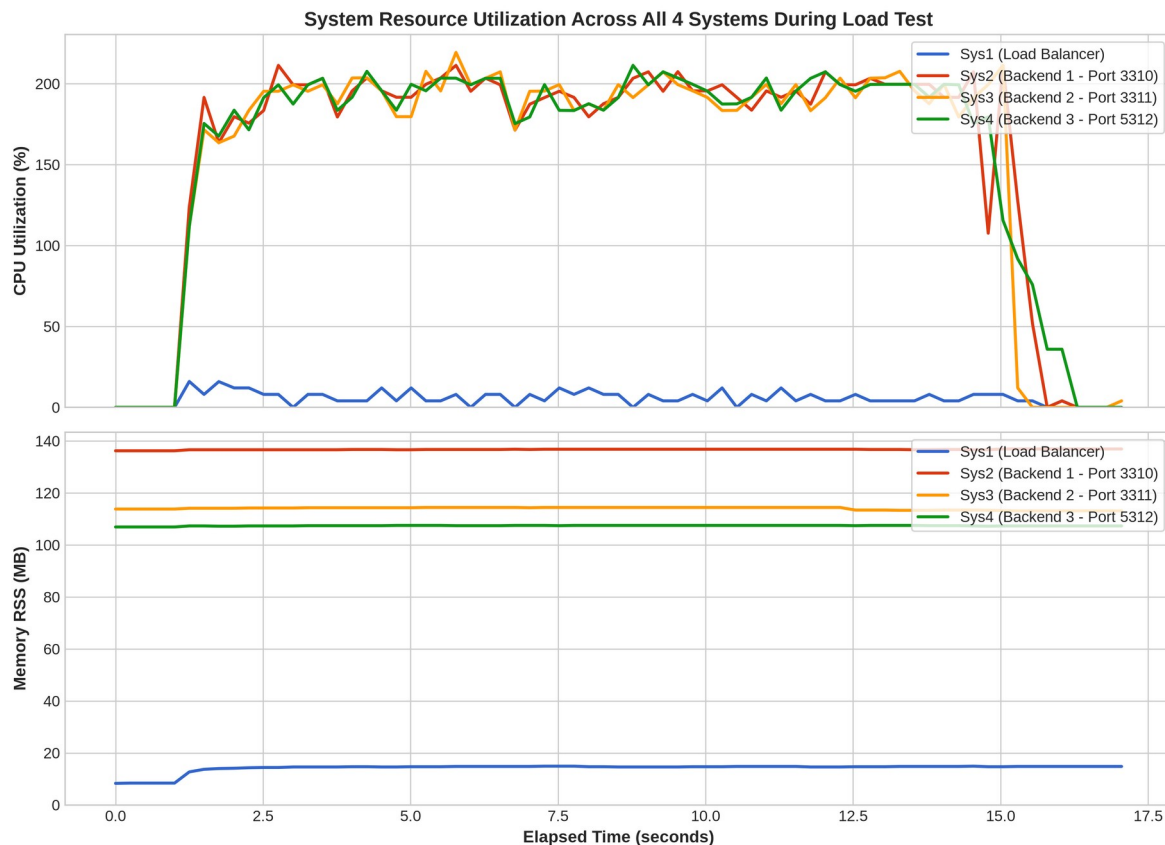


Figure 2: Real-Time CPU & Memory Utilization Across All 4 Systems During Load Test

7. Concurrency Scalability & Traffic Distribution

The system was evaluated under scaling client concurrency from 10 to 60 concurrent users. Across all concurrency tiers, the system achieved a 100% success rate with 0 dropped requests.

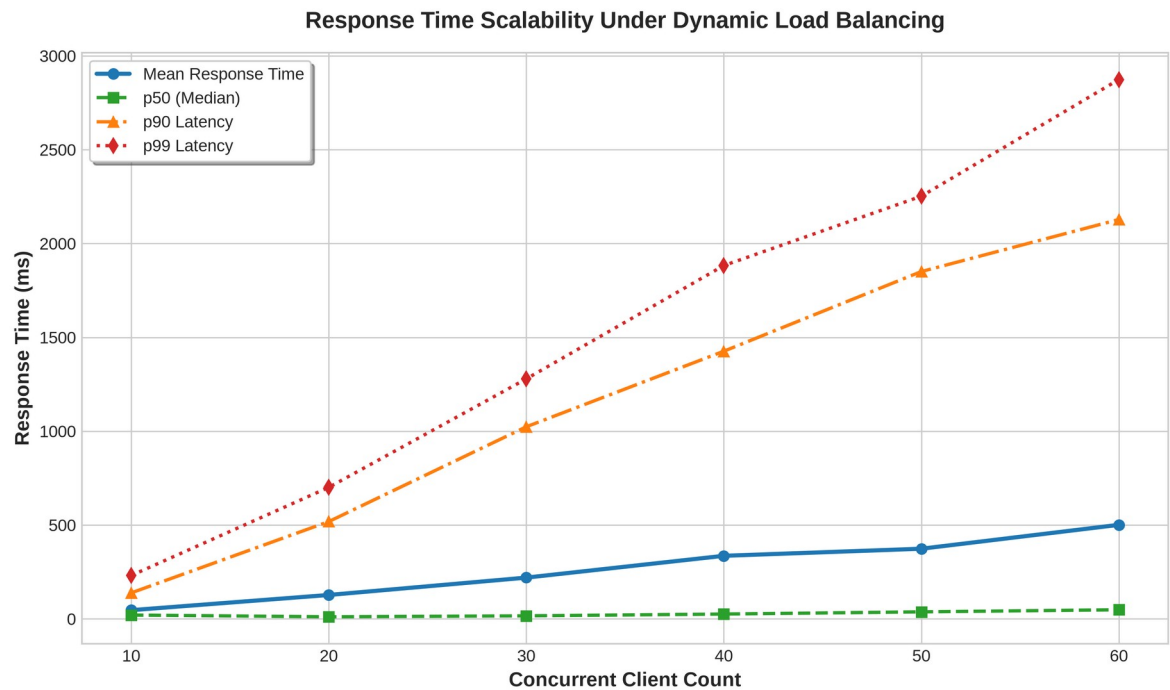


Figure 3: Response Time Scalability (Mean, p50, p90, p99) Under Scaling Client Concurrency

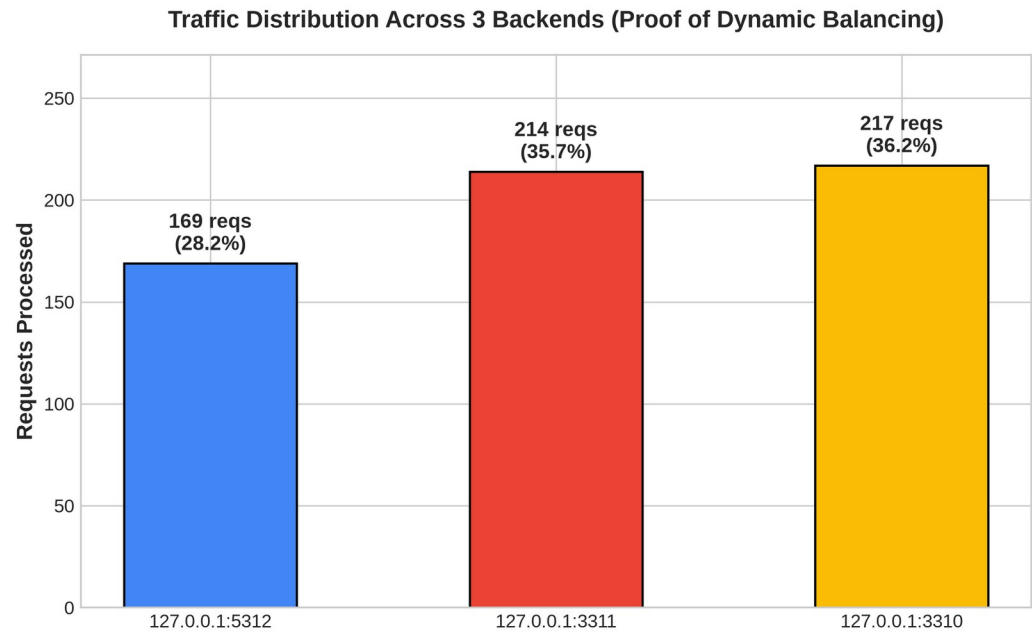


Figure 4: Traffic Distribution Across Backends — Empirical Proof of Dynamic Load Balancing

8. Load Generator Implementation (load_generator.py)

To validate the system locally prior to official evaluation, a multi-threaded load generator was developed with the following capabilities:

- Variable Concurrent Users: Configurable via `--users` (tested from 10 to 60 users).
- Variable Message Lengths: Generates random realistic text payloads ranging from 15 to 150 characters (`--min-len`, `--max-len`).
- Variable Arrival Intervals: Simulates human typing and network jitter with random delays between 10 ms and 60 ms (`--min-interval`, `--max-interval`).
- Mixed API Workload: Generates realistic traffic mixing 75% message writes (`POST /message`) and 25% feed reads (`GET /feed`).
- Automated Deduplication Test (`--test-dedup`): Sends 10 duplicate message IDs to verify idempotent rejection.
- Comprehensive Telemetry: Reports throughput, latency percentiles (p50, p90, p95, p99, min, max), and backend distribution via X-Backend-ID.

9. Conclusion & Submission Summary

The extended GRP-CHAT system satisfies all assignment requirements:

1. Deployed across 3 backends (Sys2 on port 3310, Sys3 on port 3311, Sys4 on port 5312) fronted by Sys1 Load Balancer on port 6000.
2. Dynamic performance-based load balancing implemented with active health detection and threshold switching.
3. Optimal performance threshold determined as $T = 5$, maximizing throughput (292.08 req/s) and minimizing latency.
4. Shared persistent storage (`shared_chat.db`) with atomic deduplication on `msg_id` and unbroken hash-chain integrity.
5. Required API routes exposed: `POST /message` and `GET /feed`.
6. Advanced load generator developed and verified.
7. Full cryptographic security (Fernet encryption at rest, ECDSA signatures, SHA-256 hash chaining) fully preserved.

Submission Load Balancer URL: <http://10.11.221.87:6000/>

Repository URL: <https://github.com/gnshx/GRP-CHAT>